Review 2 Report – Port Sentinel Agent

Chapter 3: System Architecture & Design

- **3.1 High-Level Architecture**
    - The implemented agent loop (run_agent() in port_sentinel_agent.py): User Prompt (CLI argument or interactive REPL input) → Local LLM via Ollama (model qwen3:4b by default, native tool-calling) → Tool Call (one of four typed Python functions) → OS-Level Execution (ss / /proc inspection, os.kill()) → structured JSON result fed back into the conversation as a tool-role message → LLM either issues another tool call or returns the final natural-language answer. This is a genuine multi-turn loop, capped at 8 iterations, not a single request/response pair — it lets the agent chain "find the owner" before "kill it" within one user request.

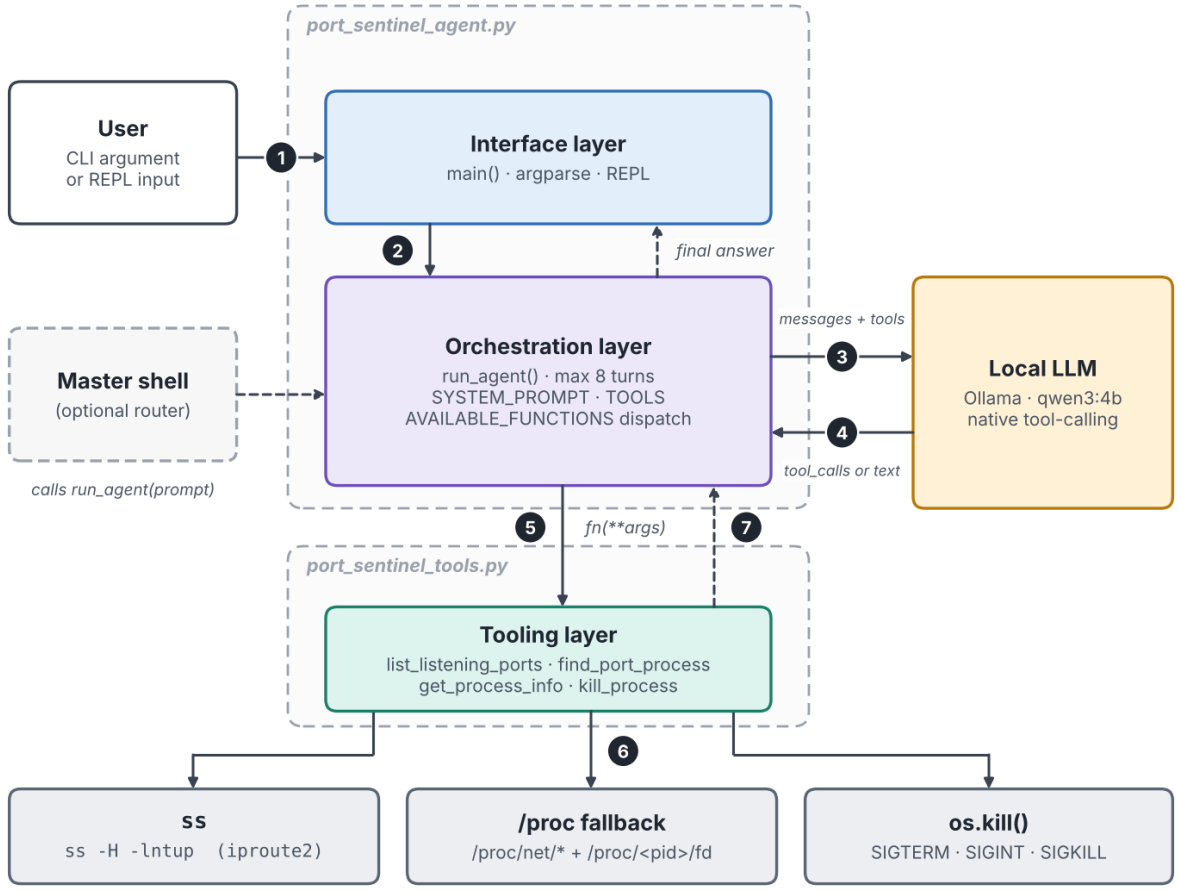


Figure 3.1 – High-level architecture of the Port Sentinel Agent. The numbered arrows follow one request from the user to the operating system and back.

Reading Figure 3.1. The two dashed frames are the project's two Python modules: `port_sentinel_agent.py` (interface and orchestration) and `port_sentinel_tools.py` (OS tools). The three grey boxes at the bottom are the only operating-system facilities the agent touches. A single request flows as follows:

1. The user's prompt enters through the interface layer, either as a one-shot CLI argument or typed into the `port-sentinel>` REPL.
2. `main()` hands the prompt to `run_agent()` in the orchestration layer. The final natural-language answer returns along the dashed arrow and is printed.
3. `run_agent()` sends the system prompt, the user prompt and the four tool definitions to the local Ollama model through `ollama.chat()`.
4. The model answers with either one or more tool calls or plain text. It never touches the operating system itself.
5. For each tool call, the orchestration layer looks the name up in `AVAILABLE_FUNCTIONS` and runs `fn(**args)`.
6. The tool function queries or signals the OS: `ss`, the `/proc` fallback, or `os.kill()`.
7. The returned dictionary is serialised to JSON and appended to the conversation as a tool-role message, so the model can see what happened. Steps 3–7 repeat until the model stops asking for tools.

The dashed **master shell** box is not part of this project. It represents the larger router described in the README, which imports `run_agent()` and forwards only port and network-process requests to it. The key design point visible in the figure is that steps 5 and 6 are executed by ordinary Python code, not by the LLM. That is where the guardrails of Section 3.3 are enforced.

Reading Figure 3.2 (below). The flowchart shows how `run_agent()` works through a request, one loop iteration at a time:

- **Setup.** The message history starts with two entries: the `SYSTEM_PROMPT` and the user's prompt.
- **Call the model.** Each iteration calls `chat(model, messages, tools=TOOLS)` and appends the reply to the history.
- **Exit on plain text.** If the reply carries no `tool_calls`, the loop returns the reply text (or "Done." if it is empty). This is the normal way for a request to finish.
- **Dispatch.** Otherwise every requested call is executed. An unknown tool name, or any exception raised inside a tool, becomes a `{"success": False, "error": ...}` result instead of crashing the agent (the red path).
- **Feed back.** Each result is appended as a `tool` message. Because the model reads it on the next iteration, it can chain actions, for example finding the owner of a port first and only then killing it.
- **Safety bound.** The loop runs at most eight times. If the model is still requesting tools after that, the function returns a fixed message, so a confused model can never loop forever.



Figure 3.2 – Control flow of run_agent(): a bounded, multi-turn tool-calling loop.

- Scope and integration: Port Sentinel is a single-domain sub-agent (TCP/UDP port discovery and process termination only), not a general-purpose shell. The README documents it as a pluggable module — a larger "master shell" router is expected to call run_agent(prompt) directly and route only port/network-process requests to it, so no shell-command string ever has to be generated by an LLM.

Listing 3.1 – Calling Port Sentinel from a master shell (pattern from the README)

```
from port_sentinel_agent import run_agent

answer = run_agent(user_prompt)
print(answer)
```

What this shows. `run_agent(prompt)` is the entire public contract of the module. The router only decides whether a request is about ports; everything after that, including tool

choice and safety checks, stays inside the agent, so no shell-command string is ever generated by an LLM.

- **3.2 Component Design**

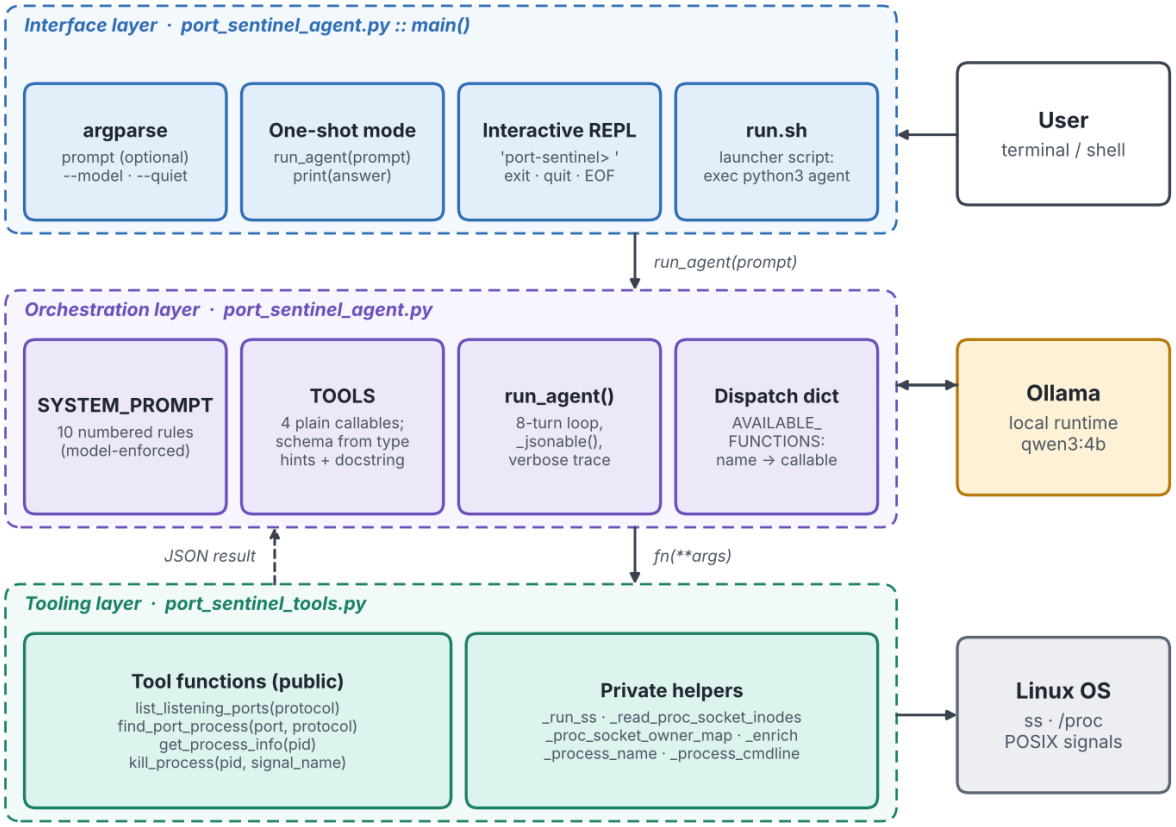


Figure 3.3 – Layered component view and module dependencies.

Reading Figure 3.3. Each band is one layer described in this section, and the boxes inside list the real functions and constants that implement it. Dependencies point downwards only: `port_sentinel_agent.py` imports the four public tool functions from `port_sentinel_tools.py`, while the tooling module knows nothing about Ollama or the prompt. That separation is why `test_tools.py` can exercise discovery and termination without any LLM running. The three boxes on the right are external systems: the terminal, the Ollama runtime and the Linux kernel interfaces.

  - **The Interface Layer:** Implemented in port_sentinel_agent.py's main() with argparse. Not a generic placeholder — it concretely supports two entry points: a one-shot CLI call (python3 port_sentinel_agent.py "<prompt>") and an interactive custom REPL ("port-sentinel> ") that loops on input() until exit/quit/EOF/Ctrl-C, plus a run.sh convenience launcher and a --quiet flag to suppress tool-trace logging.

Listing 3.2 – Interface layer: one-shot and REPL modes in `main()`

```
def main() -> int:
    # ...
    args = parser.parse_args()

    try:
        if args.prompt:
            prompt = args.prompt
        else:
            print("Port Sentinel Agent. Type 'exit' to quit.")
            while True:
                try:
                    prompt = input("port-sentinel> ").strip()
                except (EOFError, KeyboardInterrupt):
                    print()
                    return 0
                if prompt.lower() in {"exit", "quit"}:
                    return 0
                if not prompt:
                    continue
                answer = run_agent(prompt, model=args.model, verbose=not args.quiet)
                print(answer)
                print()
            return 0

        print(run_agent(prompt, model=args.model, verbose=not args.quiet))
        return 0
    except Exception as exc:
        print(f"ERROR: {exc}", file=sys.stderr)
        return 1
```

What this shows. If a prompt argument is given, the agent runs once and exits (one-shot mode). Otherwise it enters a `while True` REPL that ends on `exit`, `quit`, end-of-input or Ctrl-C. Both paths call the same `run_agent()`, and `verbose=not args.quiet` controls whether the `[LLM]` / `[OS ]` trace is printed. Any exception is reported on stderr and gives exit code 1. The omitted argparse setup defines the optional `prompt` argument and the `--model` and `--quiet` flags.

- **The Orchestration Layer:** No LangChain, LlamaIndex, or MCP client is used. Orchestration is a small, hand-written tool-calling loop against Ollama's native Python client (ollama.chat(model=..., messages=..., tools=TOOLS)). Python functions are passed directly as tools; Ollama derives each tool's JSON schema from the function's type hints and docstring, so no schema is hand-maintained. The loop reads response.message.tool_calls, dispatches through an AVAILABLE_FUNCTIONS dict, appends each result back as a tool-role message, and repeats (max 8 turns) until the model stops requesting tools.

Listing 3.3 – Orchestration layer: tools are plain Python callables, registered twice

```
TOOLS = [
    list_listening_ports,
    find_port_process,
    get_process_info,
    kill_process,
]

AVAILABLE_FUNCTIONS = {
    "list_listening_ports": list_listening_ports,
    "find_port_process": find_port_process,
    "get_process_info": get_process_info,
    "kill_process": kill_process,
}
```

What this shows. `TOOLS` is the list handed to Ollama, which derives each tool's JSON schema from the function's type hints and docstring. `AVAILABLE_FUNCTIONS` is the reverse lookup the loop uses to turn the model's tool name back into a callable. No schema is maintained by hand.

Listing 3.4 – Orchestration layer: the bounded tool-calling loop in `run_agent()`

```
def run_agent(user_prompt: str, model: str = MODEL, verbose: bool = True) -> str:
    """Run a multi-turn tool-calling loop with a local Ollama model."""
    # ...
    messages: list[dict[str, Any]] = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": user_prompt},
    ]

    for _ in range(8):
        response = chat(model=model, messages=messages, tools=TOOLS)
        messages.append(response.message)

        tool_calls = getattr(response.message, "tool_calls", None) or []
        if not tool_calls:
            return (getattr(response.message, "content", None) or "Done.").strip()

        for call in tool_calls:
            name = call.function.name
            args = dict(call.function.arguments or {})

            if verbose:
                print(f"[LLM] tool={name} args={args}")

            fn = AVAILABLE_FUNCTIONS.get(name)
            if fn is None:
                result: Any = {"success": False, "error": f"Unknown tool: {name}"}
            else:
                try:
                    result = fn(**args)
                except Exception as exc:  # keep the agent loop alive
                    result = {"success": False, "error": f"{type(exc).__name__}: {exc}"}

            if verbose:
                print(f"[OS ] {name} -> {_jsonable(result)}")

            messages.append(
                {
                    "role": "tool",
                    "tool_name": name,
                    "content": _jsonable(result),
                }
            )

    return "The agent reached its tool-call limit before producing a final response."
```

What this shows. This is the code behind Figure 3.2. Three safety properties are visible: the `for _ in range(8)` bound; the `fn is None` branch for tool names the model invented; and

the `try/except Exception` that keeps a failing tool from killing the loop. Every result is sent back as `_jsonable(result)` in a `tool`-role message.

- **The Tooling Layer:** Also not an MCP server — the "tools" are plain, in-process Python functions (port_sentinel_tools.py) called directly by the orchestration loop; there is no server/transport boundary and no file-management functionality. The four tools are list_listening_ports(protocol), find_port_process(port, protocol), get_process_info(pid), and kill_process(pid, signal_name). Port/PID discovery runs primarily through "ss -H -lntup" (regex-parsed for pid=, process name, local address, and state), with a fallback/enrichment path that reads /proc/net/{tcp,tcp6,udp,udp6} and cross-references socket inodes against /proc/<pid>/fd/* symlinks when ss cannot resolve an owner PID due to insufficient permissions.

Reading Figure 3.4 (below). `find_port_process()` first validates the port and then delegates to `list_listening_ports()`. That function trusts `ss` when it reports at least one PID. If `ss` is missing, fails, or hides the PIDs (typical for a non-root user), the code reads `/proc/net/{tcp,tcp6,udp,udp6}`, keeps TCP sockets in state `0A` (LISTEN), and resolves each socket's inode to PIDs by scanning `/proc/<pid>/fd` for `socket:[inode]` links. Back in `find_port_process()`, the rows are filtered by port, retried once through `/proc` if the match has no visible PID, and each owning PID is resolved to a process name and command line. The final dictionary (`found`, `sockets`, `processes`, `message`) is what the LLM receives.

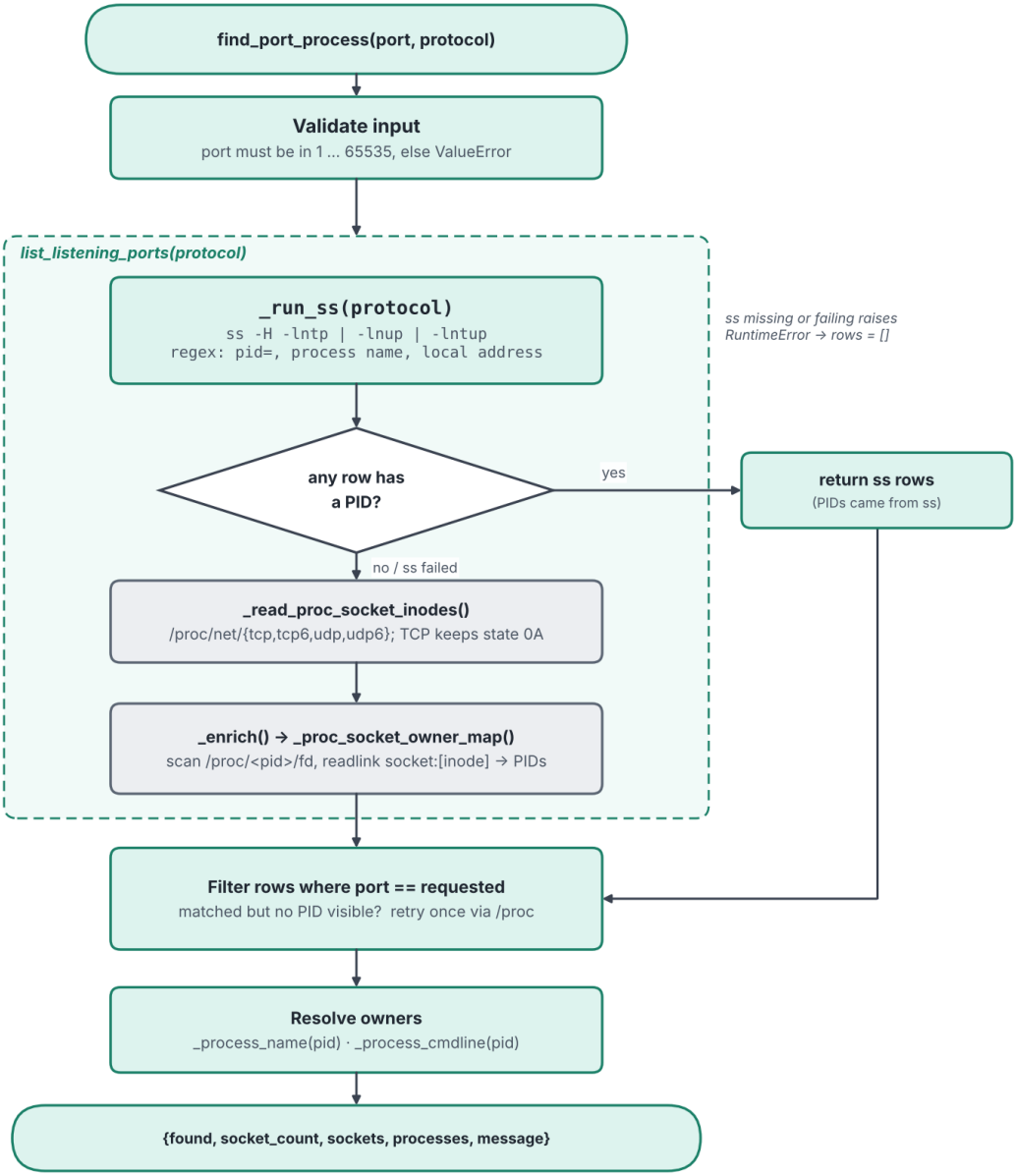


Figure 3.4 – Port-to-process discovery in `find_port_process()`: `ss` first, `/proc` as fallback.

Listing 3.5 – Tooling layer: parsing `ss` output in `_run_ss()`

```
    rows: list[dict[str, Any]] = []
    for raw in proc.stdout.splitlines():
        line = raw.strip()
        if not line:
            continue

        # Typical output:
        # LISTEN 0 4096 127.0.0.1:8080 0.0.0.0:* users:(("python3",pid=1234,fd=3))
        parts = line.split()
        if len(parts) < 5:
            continue

        local = parts[3]
        process_text = " ".join(parts[5:]) if len(parts) > 5 else ""
        pids = sorted({int(x) for x in re.findall(r"pid=(\d+)", process_text)})
        names = re.findall(r'\(\("([^"]+)"', process_text)
        port_match = re.search(r":(\d+)$", local)
        if not port_match:
            continue

        state = parts[0]
        family = "ipv6" if local.startswith("[") or ":" in local.rsplit(":", 1)[0] else
            "ipv4"
```

What this shows. The agent never executes a command built from model output. `ss` is called with a fixed argument list and its text output is parsed with regular expressions that extract the owning PIDs and process names. The port number comes from the trailing `:<port>` of the local address.

Listing 3.6 – Tooling layer: mapping socket inodes to PIDs through `/proc`

```
def _proc_socket_owner_map() -> dict[str, set[int]]:
    """Map socket inode -> PIDs by scanning /proc/<pid>/fd symlinks."""
    mapping: dict[str, set[int]] = {}
    proc_root = Path("/proc")
    for entry in proc_root.iterdir():
        if not entry.name.isdigit():
            continue
        pid = int(entry.name)
        fd_dir = entry / "fd"
        try:
            fds = list(fd_dir.iterdir())
        except (PermissionError, FileNotFoundError):
            continue
        for fd in fds:
            try:
                target = os.readlink(fd)
            except (PermissionError, FileNotFoundError, OSError):
                continue
            match = re.fullmatch(r"socket:\[(\d+)\]", target)
            if match:
                mapping.setdefault(match.group(1), set()).add(pid)
    return mapping
```

What this shows. The fallback walks every numeric directory in `/proc`, reads the symbolic links in `/proc/<pid>/fd`, and records which PIDs hold a `socket:[inode]` link. `_enrich()` then joins this map with the sockets parsed from `/proc/net/*`. Unreadable processes are skipped through the `PermissionError` handling, so a non-root run may not see sockets owned by other users.

Listing 3.7 – Tooling layer: the `ss`-first, `/proc`-fallback decision

```
def list_listening_ports(protocol: str = "all") -> list[dict[str, Any]]:
    # ...
    protocol = protocol.lower()
    if protocol not in {"tcp", "udp", "all"}:
        raise ValueError("protocol must be tcp, udp, or all")
```

```
    try:
        rows = _run_ss(protocol)
        # ss is the preferred source. It may omit PIDs when permissions are
        # insufficient, so supplement missing owners from /proc.
        if any(row["pids"] for row in rows):
            return rows
    except RuntimeError:
        rows = []

    proc_rows = _enrich(_read_proc_socket_inodes(protocol))
    return proc_rows
# ...
def find_port_process(port: int, protocol: str = "tcp") -> dict[str, Any]:
    # ...
    rows = list_listening_ports(protocol)
    matches = [row for row in rows if row["port"] == port]

    # A non-root `ss -p` may show the socket but hide its PID. For a specific
    # port, retry through /proc so the tool can still resolve the owner when
    # the kernel permissions allow the /proc fd scan.
    if matches and not any(row.get("pids") for row in matches):
        proc_rows = _enrich(_read_proc_socket_inodes(protocol))
        proc_matches = [row for row in proc_rows if row["port"] == port]
        if proc_matches:
            matches = proc_matches
```

What this shows. The first block is the fallback decision from Figure 3.4: `ss` rows are returned if any of them carries a PID, and a `RuntimeError` from `_run_ss()` (for example `ss` not installed) is swallowed so the `/proc` path takes over. The second block is the extra retry inside `find_port_process()` for the case where a port matched but no PID was visible.

- **3.3 Security & Guardrail Framework**

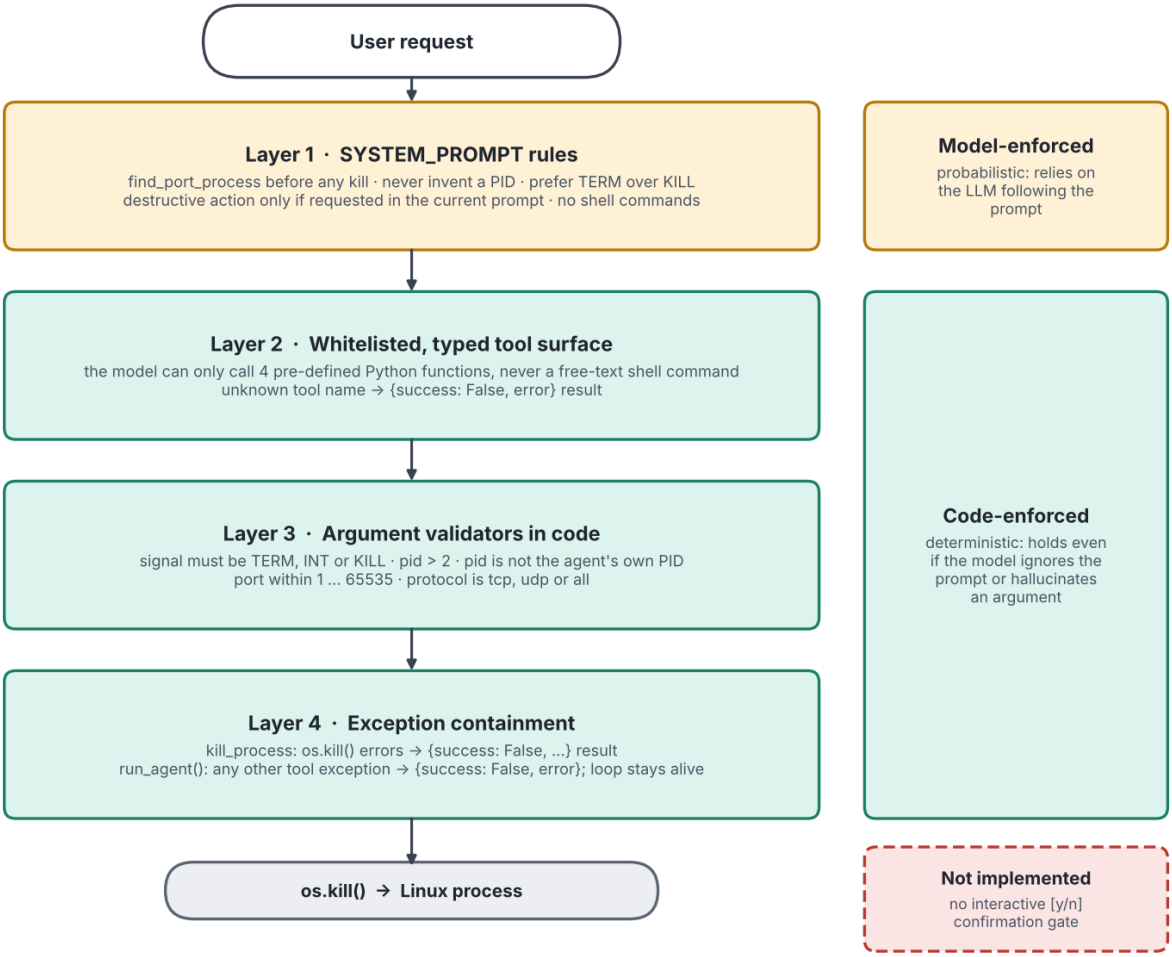


Figure 3.5 – Four layers of defence between a user request and a process signal.

Reading Figure 3.5. A request passes four checkpoints before any signal is sent. Layer 1 is advisory: the rules in `SYSTEM_PROMPT` steer the model but cannot force it. Layers 2 to 4 are deterministic Python and hold even if the model ignores its instructions or invents an argument. Because Layer 1 is the only place where "act only on an explicit request" is enforced, the dashed red box marks the gap already noted in this section: there is no code-level interactive confirmation (an "Are you sure? [y/n]" prompt) before `kill_process` runs.

  - "Human-in-the-loop" (as implemented): there is no secondary interactive confirmation prompt (no "Are you sure? [y/n]") in the code. The guardrail is enforced entirely through prompt engineering in SYSTEM_PROMPT: a destructive action must be directly requested in the current user turn or be an unambiguous consequence of it (rule 10); the agent must resolve the real PID via find_port_process before any kill and must never invent one (rules 1 and 3); and it must prefer SIGTERM, escalating to SIGKILL only on explicit request or after TERM fails (rule 2). This is a model-enforced, not a code-enforced, confirmation step — a gap to flag if a hard confirmation gate is required by the rubric.

Listing 3.8 – Layer 1: the `SYSTEM_PROMPT` rules that act as a model-enforced confirmation step

```
SYSTEM_PROMPT = """
You are Port Sentinel, a Linux network-port management agent.

Your job is to answer natural-language requests about TCP/UDP ports and the
processes that own them, using ONLY the provided tools for OS inspection and
process management.

Rules:
1. For a specific port, call find_port_process first. Do not guess a PID.
2. If the user asks to stop/kill/terminate the process, call kill_process with
the PID returned by find_port_process. Prefer TERM; use KILL only when the user
explicitly requests force kill or when the user asks again after TERM fails.
3. Never kill PID 1, the Port Sentinel Agent itself, or invent a PID.
4. If multiple processes own the requested port, explain that and choose only
processes clearly matching the user's request; do not blindly kill every PID.
5. If the user only asks what is using a port, inspect and report without killing.
6. Assume TCP when the user says only "port <number>". Ask for or infer UDP only
when their wording clearly refers to UDP/datagrams. For "both", inspect both.
7. You may call list_listening_ports for requests such as "show listening ports".
8. Be concise but include port, protocol, PID, process name/cmdline, and action taken.
9. Never generate a shell command for the user as a substitute for a tool call.
10. A destructive action must be directly requested by the user in the current
prompt or be an unambiguous consequence of a direct request such as "What is
using 8080? Stop it."
""".replace("\n", " ")
```

What this shows. Rules 1 to 3 force the "find the PID first, never invent one" workflow, rule 2 prefers `TERM` over `KILL`, rule 9 forbids generating shell commands, and rule 10 ties any destructive action to the current user turn. The final `.replace("\n", " ")` flattens the prompt into a single line before it is sent to the model.

- Command sanitization / blacklisting (as implemented): there is no regex blacklist of destructive shell strings such as "rm -rf /", because the design removes that attack surface architecturally rather than filtering it — the LLM can never submit a free-text shell command at all; it may only invoke one of four pre-defined Python functions with typed, validated arguments (SYSTEM_PROMPT rule 9: "Never generate a shell command for the user as a substitute for a tool call"). The real guardrails are argument-level validators inside port_sentinel_tools.py: kill_process accepts only TERM, INT, or KILL; refuses pid ≤ 2 (protects PID 1/init and other low system PIDs); and refuses pid == os.getpid() (protects the agent's own process). find_port_process rejects ports outside 1–65535; list_listening_ports rejects any protocol other than tcp/udp/all; and PermissionError/ProcessLookupError from os.kill() are caught and returned as structured {"success": False, ...} results instead of raising and crashing the agent loop.

Reading Figure 3.6 (below). Every call to `kill_process()` walks through the following checks:

- **Rejected before any signal (red boxes, right).** Four checks run in order: the signal name (`TERM`, `INT` or `KILL` only), a PID above 2, a PID different from the agent's own, and the process still existing.

- **Exceptions versus results.** The first three checks raise `ValueError` or `PermissionError`; `run_agent()` catches them and returns `{"success": False, "error": ...}` to the model. The existence check returns a structured result directly.
- **Failures during the signal.** `os.kill()` can still raise `ProcessLookupError` (the process exited in the meantime) or `PermissionError` (it belongs to another user). Both are converted into `{"success": False, ...}`, with a hint to re-run with sufficient privileges.
- **Success.** Only when every check passes does the function return `{"success": True, ...}` with the signal, process name and command line.

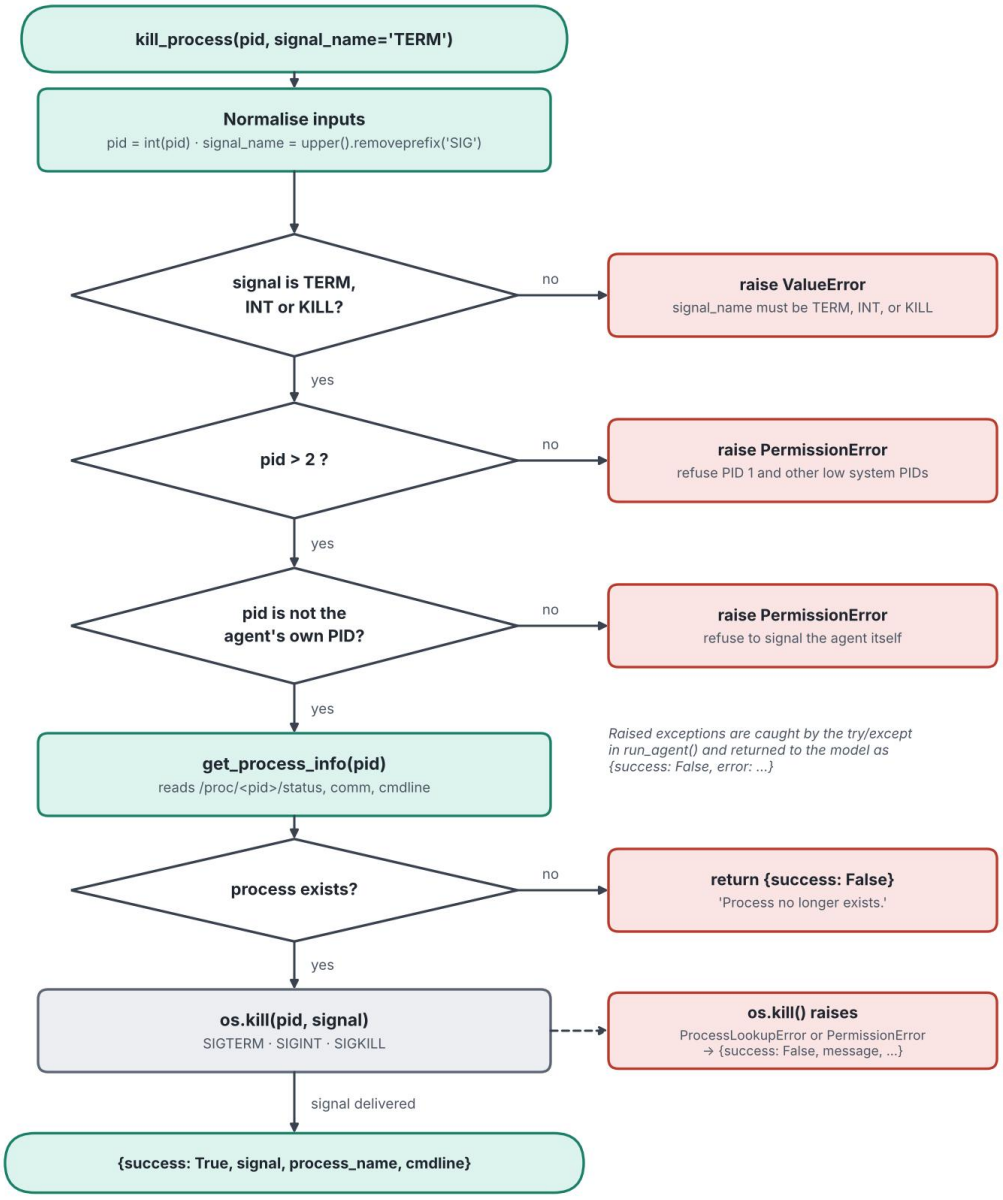


Figure 3.6 – Validation sequence inside `kill_process()` (Layers 3 and 4).

Listing 3.9 – Layers 3 and 4: validators and error handling in `kill_process()`

```
def kill_process(pid: int, signal_name: str = "TERM") -> dict[str, Any]:
    # ...
    pid = int(pid)
```

```
    signal_name = signal_name.upper().removeprefix("SIG")

    allowed = {"TERM": signal.SIGTERM, "INT": signal.SIGINT, "KILL": signal.SIGKILL}
    if signal_name not in allowed:
        raise ValueError("signal_name must be TERM, INT, or KILL")
    if pid <= 2:
        raise PermissionError("Refusing to signal PID 1 or other very low system PIDs.")
    if pid == os.getpid():
        raise PermissionError("Refusing to signal the Port Sentinel Agent itself.")

    info_before = get_process_info(pid)
    if not info_before["exists"]:
        return {"success": False, "pid": pid, "message": "Process no longer exists."}

    try:
        os.kill(pid, allowed[signal_name])
    except ProcessLookupError:
        return {"success": False, "pid": pid, "message": "Process exited before the
            signal was sent."}
    except PermissionError as exc:
        return {
            "success": False,
            "pid": pid,
            "process_name": info_before.get("process_name"),
            "message": "Permission denied. Re-run the agent with sufficient privileges
                (usually sudo).",
            "error": str(exc),
        }
```

What this shows. The allow-list `{"TERM", "INT", "KILL"}` replaces any blacklist of dangerous strings: a signal outside the list, a PID of 2 or below, or the agent's own PID is refused in code, whatever the model asked for.

Listing 3.10 – Layer 3: input validation in the read-only tools

```
def list_listening_ports(protocol: str = "all") -> list[dict[str, Any]]:
    # ...
    protocol = protocol.lower()
    if protocol not in {"tcp", "udp", "all"}:
        raise ValueError("protocol must be tcp, udp, or all")
# ...
def find_port_process(port: int, protocol: str = "tcp") -> dict[str, Any]:
    # ...
    port = int(port)
    if not 1 <= port <= 65535:
        raise ValueError("port must be between 1 and 65535")
```

What this shows. `list_listening_ports()` accepts only `tcp`, `udp` or `all`, and `find_port_process()` accepts only ports 1 to 65535. Invalid values raise `ValueError`, which the loop in `run_agent()` converts into an error result for the model.

Chapter 4: Implementation Methodology

- **4.1 Development Environment & Tech Stack**

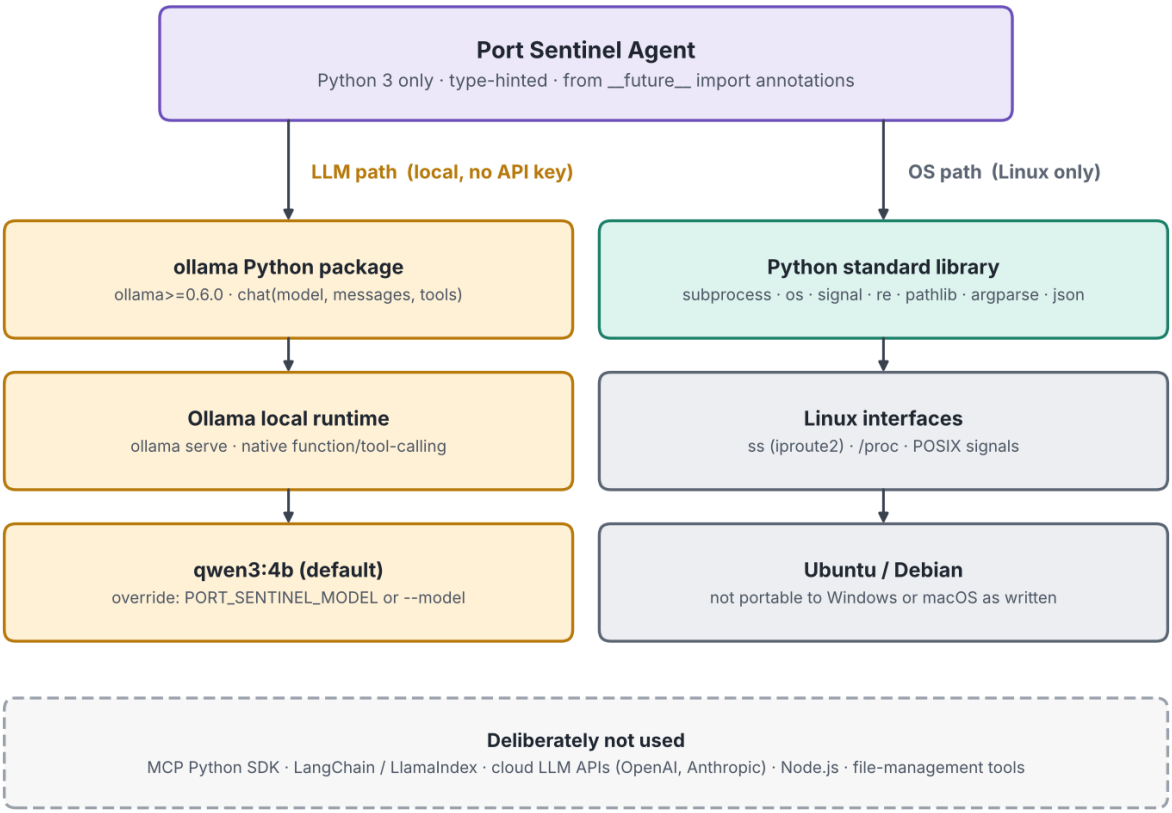


Figure 4.1 – Technology stack and runtime dependencies.

Reading Figure 4.1. The agent is a single Python 3 program with two independent dependency chains. On the left, the LLM path runs from the `ollama` Python package through the local Ollama runtime to the `qwen3:4b` model; no cloud API or key is involved. On the right, the OS path uses only the Python standard library to reach Linux interfaces (`ss`, `/proc`, POSIX signals), which is why the agent is specific to Ubuntu/Debian. The dashed box lists technologies that appear in similar projects but are deliberately absent here.

- **Programming Language:** Python 3 only (type-hinted throughout, “from __future__ import annotations”); there is no Node.js component anywhere in the project.
- **LLM Provider:** Local-only, via Ollama — ollama.chat(), default model qwen3:4b, overridable with the PORT_SENTINEL_MODEL environment variable or the --model flag. No cloud LLM (OpenAI API / Anthropic Claude) is wired in; the README frames this as a deliberate “local, no API key, class-demo-safe” choice.

Listing 4.1 – Model selection and the optional `ollama` import

```
try:
    from ollama import chat
except ImportError:
    chat = None  # type: ignore[assignment]
# ...
MODEL = os.getenv("PORT_SENTINEL_MODEL", "qwen3:4b")
    # ...
    if chat is None:
        raise RuntimeError(
            "Python package 'ollama' is missing. Run: python3 -m pip install -r
                requirements.txt"
        )
```

What this shows. The default model is `qwen3:4b`, overridable through the `PORT_SENTINEL_MODEL` environment variable (and the `--model` flag). The `ollama` import is wrapped in `try/except ImportError`, so the module can still be imported without the package and fails with a clear installation hint only when `run_agent()` is actually called.

- **SDKs:** The ollama Python package (>=0.6.0, pinned in requirements.txt), used for its native function/tool-calling support. The MCP Python SDK is not used anywhere — the tools are plain Python callables, not an MCP server/client pair.

Listing 4.2 – Dependencies and setup (commands from the README)

```
# Ubuntu prerequisites (ss comes from iproute2)
sudo apt install -y python3 python3-venv iproute2 curl

# Local model served by Ollama
ollama pull qwen3:4b

# Python environment; requirements.txt contains: ollama>=0.6.0
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
```

What this shows. The whole Python dependency set is a single package. Everything else is either the standard library or a system tool.

- **OS / system dependency:** Ubuntu/Debian Linux specifically — iproute2 for the ss command, the /proc filesystem for the fallback discovery path, and POSIX signals (os.kill) for process control. The agent is not cross-platform (it will not run its core tools on Windows or macOS as written).

Listing 4.3 – Failing clearly when `ss` is unavailable

```
    try:
        proc = subprocess.run(
            cmd,
            check=False,
            capture_output=True,
            text=True,
            timeout=5,
        )
    except FileNotFoundError as exc:
        raise RuntimeError("The 'ss' command is not installed. Install iproute2.") from
            exc

    if proc.returncode != 0:
        raise RuntimeError(proc.stderr.strip() or "ss failed")
```

What this shows. `ss` is invoked with a fixed argument list and a 5-second timeout. A missing binary becomes a readable `RuntimeError` that points to the iproute2 package, and

`list_listening_ports()` catches that error and falls back to `/proc`. Both mechanisms exist only on Linux, which is why the tools do not run on Windows or macOS as written.

- **4.2 Step-by-Step Implementation**
    - **Step 1:** Built the CLI entry point — the argparse-based main() in port_sentinel_agent.py, supporting a one-shot prompt argument and a "port-sentinel>" interactive REPL loop, plus run.sh and a run_agent(prompt) function meant to be imported by a higher-level "master shell" router.

Listing 4.4 – Launcher script and the two ways to start the agent

```
#!/usr/bin/env bash
set -euo pipefail
cd "$(dirname "$0")"
exec python3 port_sentinel_agent.py "$@"
# one-shot mode
python3 port_sentinel_agent.py "What process is using port 8080?"

# interactive REPL (type exit or quit to leave)
./run.sh
port-sentinel> What is listening on port 8080?
```

What this shows. `run.sh` simply changes into its own directory and forwards all arguments to the agent, so `./run.sh "<prompt>"` and `python3 port_sentinel_agent.py "<prompt>"` are equivalent. Without an argument, the same script opens the REPL.

- **Step 2:** Built the four OS tool functions directly in Python (no MCP server) in port_sentinel_tools.py — list_listening_ports, find_port_process, get_process_info, kill_process — backed by ss parsing with a /proc-based fallback for PID resolution, plus demo_server.py (a disposable local TCP listener used only for safe testing) and test_tools.py (an automated discover → inspect → SIGTERM integration test) to exercise the tools without any LLM in the loop.

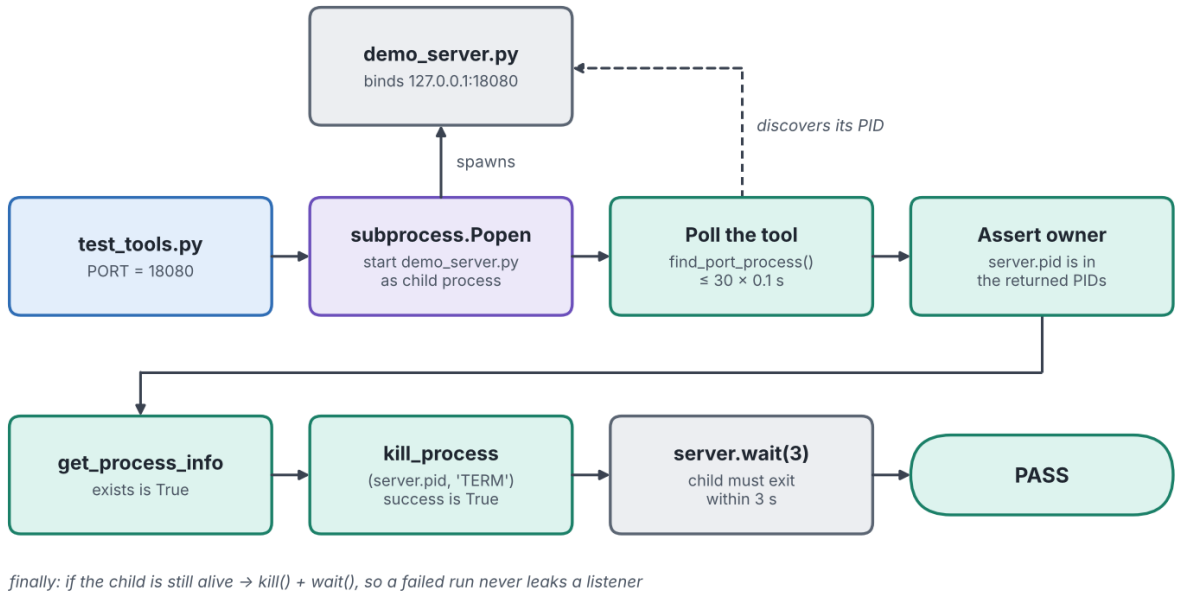


Figure 4.3 – Automated tool test (`test_tools.py`): discover, inspect and terminate, with no LLM in the loop.

Reading Figure 4.3. The test starts the disposable `demo_server.py` as a child process, polls `find_port_process()` until the child's PID appears, asserts that it is the owner, checks it with `get_process_info()`, sends `SIGTERM` through `kill_process()` and finally waits for the child to exit. A `finally` block kills any leftover child, so even a failing run does not leave a listener behind.

Listing 4.5 – Disposable test listener (`demo_server.py`)

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind(("127.0.0.1", args.port))
    server.listen(5)
    print(f"Demo server listening on 127.0.0.1:{args.port}", flush=True)
    while True:
        conn, addr = server.accept()
        with conn:
            conn.sendall(b"Port Sentinel demo server\n")
```

What this shows. A minimal TCP server bound to `127.0.0.1` that answers every connection with a fixed line. It gives the tools a safe, harmless process to discover and terminate.

Listing 4.6 – Integration test of the OS tools (`test_tools.py`)

```
PORT = 18080
# ...
def main() -> int:
    server = subprocess.Popen(
        [sys.executable, "demo_server.py", str(PORT)],
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True,
    )
    try:
        # Wait until the child has bound the port.
        for _ in range(30):
            result = find_port_process(PORT, "tcp")
            if result["processes"]:
                break
            time.sleep(0.1)
        else:
            raise AssertionError(f"Could not discover PID for test port {PORT}: {result}")
```

```
        pids = {p["pid"] for p in result["processes"]}
        assert server.pid in pids, (server.pid, result)
        info = get_process_info(server.pid)
        assert info["exists"] is True

        kill_result = kill_process(server.pid, "TERM")
        assert kill_result["success"] is True, kill_result
        server.wait(timeout=3)
        print("PASS: discover -> inspect -> SIGTERM")
        return 0
    finally:
        if server.poll() is None:
            server.kill()
            server.wait()
```

What this shows. The `for ... else` loop waits up to three seconds (30 × 0.1 s) for the child to bind its port and raises an `AssertionError` if its PID is never discovered. The assertions then check ownership, process existence and the kill result. On success the script prints `PASS: discover -> inspect -> SIGTERM`.

- **Step 3:** Prompt-engineered SYSTEM_PROMPT with 10 explicit numbered rules governing tool-call ordering, PID safety, protocol defaulting (assume TCP unless the user's wording clearly means UDP), and signal escalation. “Exact tool matching” / JSON format enforcement is delegated to Ollama's native tool-calling (schemas are auto-derived from each function's type hints and docstring) rather than hand-written as a separate validation layer.

Listing 4.7 – A tool definition: the signature and docstring are the schema

```
def kill_process(pid: int, signal_name: str = "TERM") -> dict[str, Any]:
    """Send a signal to a Linux process.

    Args:
        pid: PID to signal. The agent should obtain this from find_port_process.
        signal_name: TERM (recommended), INT, or KILL.
    """
```

What this shows. There is no hand-written JSON schema. Ollama reads the parameter names, the type hints (`pid: int`, `signal_name: str`), the default value and the docstring, including its `Args:` section, to describe the tool to the model. The prompt rules in Listing 3.8 tell the model how to use it.

- **Step 4:** Implemented the execution wrapper and output formatter inside run_agent(): a bounded (8-iteration) tool-call loop; an AVAILABLE_FUNCTIONS dispatch dict wrapped in try/except around every call so a tool exception becomes a structured {"success": False, "error": ...} result instead of crashing the agent; a _jsonable() helper that serializes results for the tool-role messages sent back to the model; and an optional verbose trace ([LLM] tool=... args=... / [OS] tool -> ...) for grading/demo visibility, toggled off with --quiet.

Listing 4.8 – Result serialisation and the verbose trace

```
def _jsonable(value: Any) -> str:
    return json.dumps(value, ensure_ascii=False, default=str)
            # ...
            if verbose:
                print(f"[LLM] tool={name} args={args}")
            # ...
            if verbose:
                print(f"[OS ] {name} -> {_jsonable(result)}")
```

What this shows. `_jsonable()` serialises any tool result; `default=str` means an unusual value can never raise. The two `print` calls produce the `[LLM]` and `[OS ]` trace lines that make each decision visible during a demo, and `--quiet` sets `verbose=False` to suppress them.

Reading Figure 4.2 (below). The request takes three loop iterations. In iteration 1 the model asks for `find_port_process`, and the tool resolves the owning PID through `ss` or `/proc`. In iteration 2 the model, now holding the PID, asks for `kill_process(pid, 'TERM')` and the tool sends `SIGTERM`. In iteration 3 the model sees `{"success": true}` and replies in plain text, which ends the loop. Solid arrows are calls and dashed arrows are returns.

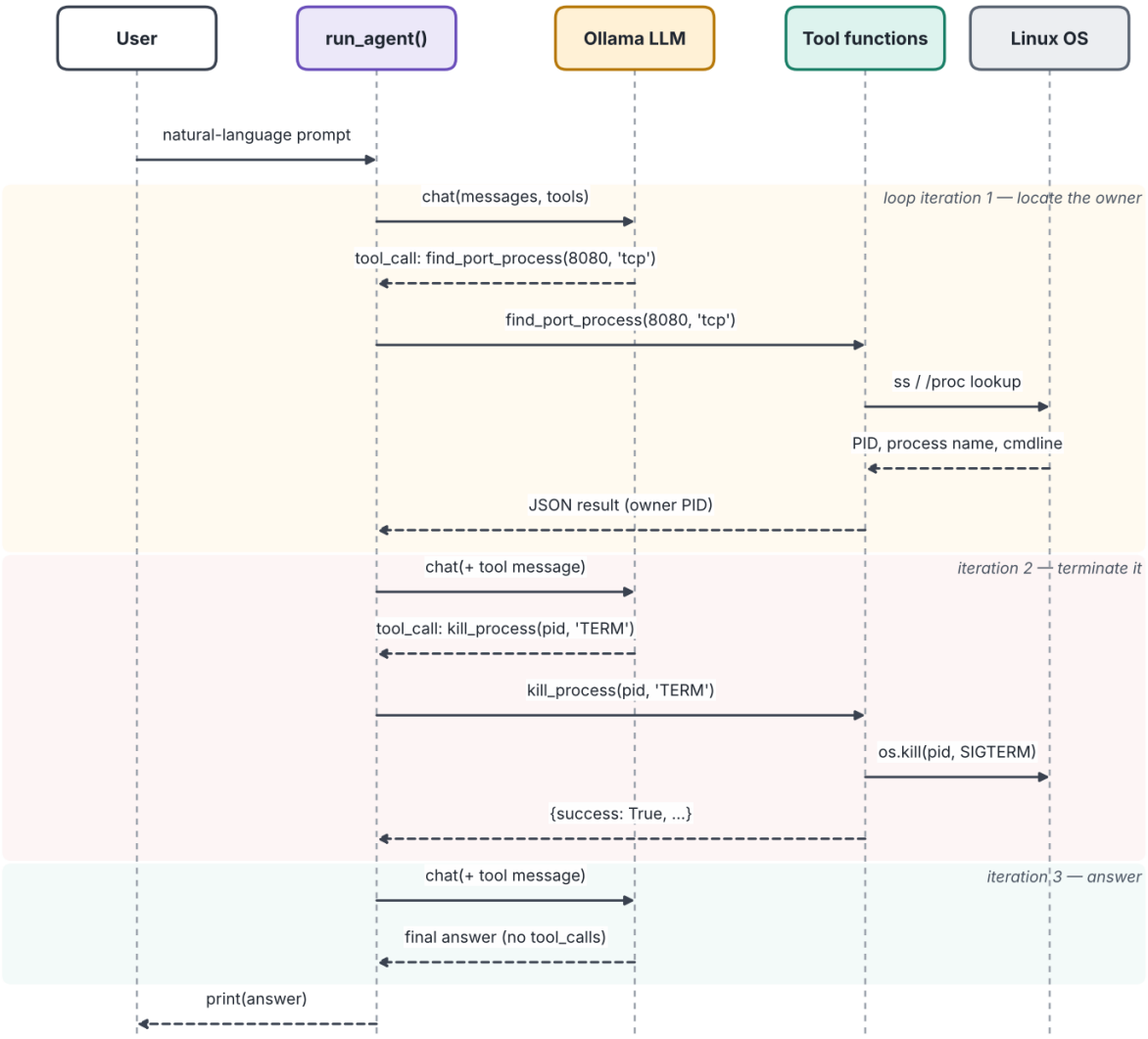


Figure 4.2 – Sequence of one complete request: “What is using port 8080? Stop it.”

Listing 4.9 – Captured trace of the loop for the request above

```
[LLM] tool=find_port_process args={'port': 8080, 'protocol': 'tcp'}
[OS ] find_port_process -> {"port": 8080, "protocol": "tcp", "found": true,
    "socket count": 1, "sockets": [{"protocol": "tcp", "state": "LISTEN",
    "local_address": "*:8080", "port": 8080, "family": "ipv4", "pids": [522],
    "process_names": ["python3"], "source": "proc"}], "processes": [{"pid": 522,
    "process_name": "python3", "cmdline": "/usr/bin/python3 demo_server.py 8080"}],
    "message": "Found 1 owning process(es) for port 8080."}
[LLM] tool=kill_process args={'pid': 522, 'signal_name': 'TERM'}
[OS ] kill process -> {"success": true, "pid": 522, "signal": "SIGTERM", "process name":
    "python3", "cmdline": "/usr/bin/python3 demo_server.py 8080", "message": "Sent
    SIGTERM to PID 522."}
TCP port 8080 was held by python3 (demo_server.py, PID 522). I sent SIGTERM and the
    process has stopped.
```

How this trace was produced. Ollama is not installed in the environment used to prepare this report, so the model's replies were scripted: a stand-in for `ollama.chat` issued the two tool calls shown. Everything on the `[OS ]` lines is real output from `find_port_process()` and `kill_process()` run against a live `demo_server.py` on port 8080. Long lines are wrapped here, and `"source": "proc"` appears because `ss` is not installed in that environment, which exercises the `/proc` fallback. With a real model the wording and the exact calls may differ.